

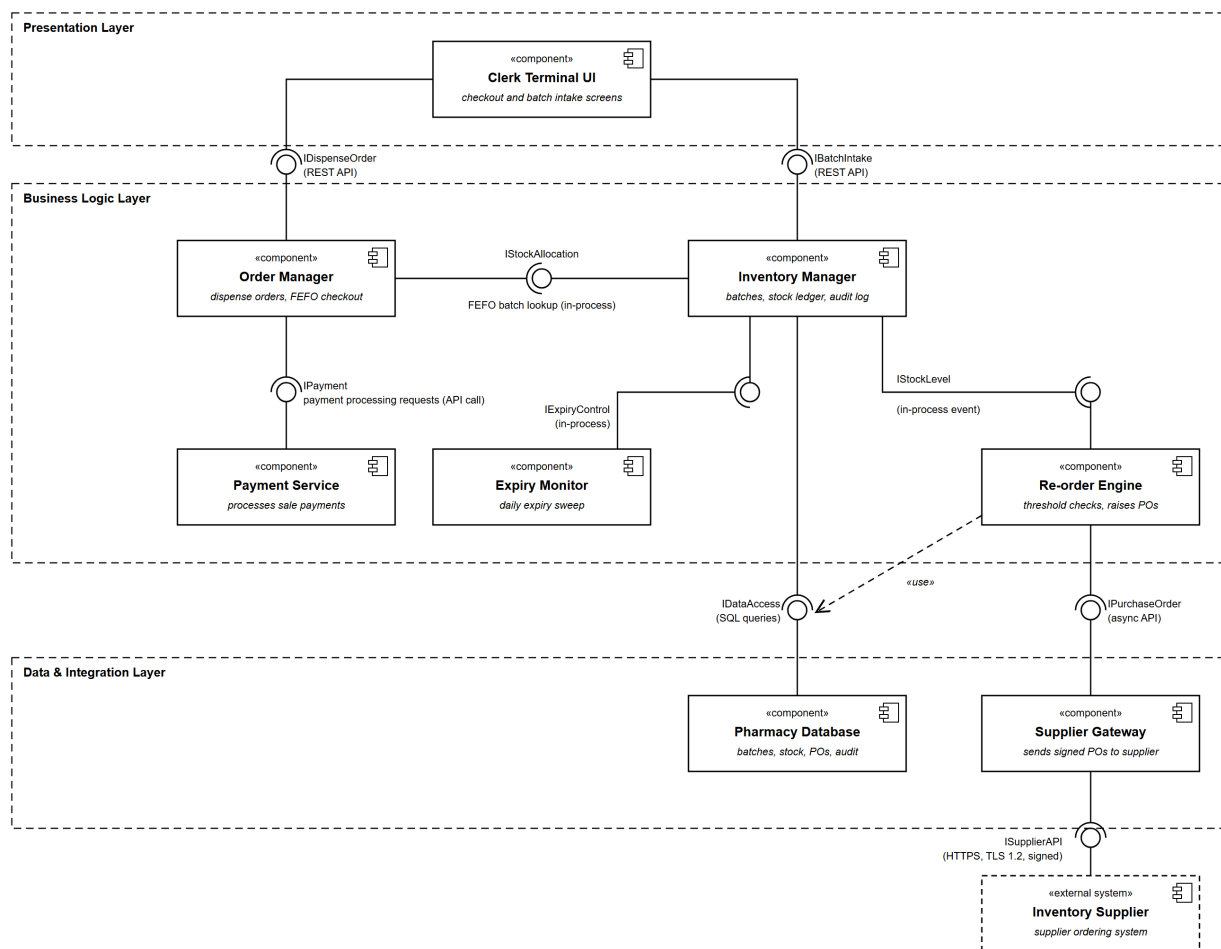
Software Engineering

Lab 3 - Component Modelling and Architectural Pattern Selection

Name : Rohan Suresh
SRN : PES1UG24CS383
Problem Statement : #15 - Pharmacy Expiry & Re-order Dispatch Engine
Repository : <https://github.com/RohanMay-Suresh/SE-Labs-PES1UG24CS383>

1. Component Diagram

The components are grouped into three layers. Order Manager and Payment Service were given; the rest come from the Lab 1 use cases. The supplier's system is external.



2. Architecture Justification

Architecture Selection: We chose Layered Architecture for the Pharmacy Expiry & Re-order Dispatch Engine.

Architectural Choice: Layered, with a presentation layer (clerk screens), a business logic layer (orders, payment, inventory, expiry and re-ordering) and a data and integration layer (database and supplier gateway).

Reason 1: Every dispense has to take stock from the nearest-expiry batch and write an audit entry together, all or nothing. With one business layer and one database this runs as a single transaction, which would be much harder across separate microservices.

Reason 2: The FEFO, expiry and threshold rules are needed by checkout, the daily sweep and re-ordering. Keeping them in the business layer means they are written once, and since it is a single pharmacy there is no need to scale parts separately.

Security Advantage: The clerk UI cannot reach the database directly. Every request goes through the business layer, which checks permissions and writes the audit entry, and supplier traffic only leaves through the Supplier Gateway over TLS.

Performance Benefit: Checkout uses in-process calls between layers instead of network calls between services, so FEFO lookup and stock deduction stay fast. Purchase orders are sent asynchronously, so they don't slow checkout down.